

Algorithm Design - First Midterm
23-04-2026, Time: 120 minutes
Prof. Stefano Leonardi, Dr. Simone Fioravanti

Instructions:

- Before you start, write on the top of each sheet (1) your full name, (2) your student ID (*matricola*) and (3) the sheet number (1, 2, 3, and 4).
- If you use additional sheets as rough draft (*brutta*) for calculations, etc., you must hand it, along with the rest of the exam. You can keep the question sheet.
- The rooms are small so it may be tempting to copy. Note that if we catch you copying in any way (copying from another student, using your phone, etc.) you will be automatically disqualified.

Problem 1

(10 points) Consider the problem of choosing **cell phone tower locations** on a long straight road. We are giving a list of distances along the road at which there are customers' houses, say $\{x_i\}_{i=1}^n$. The houses' locations are listed in order of increasing distance, i.e., $x_i \leq x_{i+1}, i = 1, \dots, n-1$. No towers have yet been built (so no customers currently have service). A survey of the road has provided a set of J possible tower locations, $\{t_j\}_{j=1}^J$, where these potential tower locations are also measured in terms of the distance along the road. The towers' locations are also listed in order of increasing distance, i.e., $t_j \leq t_{j+1}, j = 1, \dots, J$. (You can assume that all these distances are integers.) Each customer will get service (at home) if and only if they are within a range $R > 0$ of at least one cell phone tower that gets built, that is, the house at x_i will have service iff there exists a tower that has been built at some t_j with $|x_i - t_j| \leq R$. **You can assume that if all the towers were built, then every home would get service.** However, such a solution could be overly expensive for the phone company. How can we minimize the number of towers that need to be built but still provide service at every house?

1. Describe an algorithm that finds the minimum number of towers required along with a set of locations for those towers;
2. Prove the correctness of the proposed algorithm;
3. Discuss its running time.

Problem 2

(10 points) There are n families who go to a resort together for a vacation. Family i consists of x_i people. There are m activities available at the resort, and at most y_j people are allowed to participate in activity j . As the trip planner, your job is to assign people from all families to activities. But you need to make sure that each person from each family is assigned to exactly one activity, no activity j is assigned more than y_j people, and no two people from the same family are assigned to the same activity. The problem is to design a polynomial-time algorithm that either produces a feasible assignment of people to activities or declares that no such assignment exists.

1. Describe a reduction of this problem to a network flow one;
2. Prove that assessing the existence of a feasible planning is equivalent to a condition on the max-flow of the network;
3. Discuss the running time of the procedure that uses the above reduction to solve the activity planning's problem.

Problem 3

(10 points) Consider a directed graph $G = (V, E)$ where each edge $e = (u, v)$ is associated to a weight $c_{uv} \in \mathbb{R}$. Assume for simplicity that the graph is strongly connected (meaning that there exists a path from any node in V to any other one). The problem is to devise an algorithm that decides whether G contains a **negative cycle** or not.

1. Recall the definition of the subproblems $OPT[i, v]$ employed by the Bellman-Ford algorithm and the recurrence equation relating them.
2. Describe a dynamic programming algorithm that decides the existence of a negative cycle.
3. Discuss the time and space complexity of the proposed solution.

Problem 4 (Bonus)

(5 points)

Consider a seller who receives a sequence of n offers $\{b_t\}_{t=1}^n$ for items on sale from n buyers at each time $t = 1, \dots, n$. The seller knows the values of the offers in advance, and she must decide on a sequence of non-increasing prices $\{p_t\}_{t=1}^n$, that is $p_t \geq p_{t+1}$ for all $t = 1, \dots, n-1$. If $p_t \leq b_t$ the buyer accepts to buy the item at time t and the seller receives a profit p_t . On the other hand, if $p_t > b_t$, the seller has 0 profit at time t . The overall profit of the seller is therefore equal to

$$\sum_{t=1}^n p_t \mathbb{1}(b_t \geq p_t),$$

where the indicator function $\mathbb{1}(\cdot)$ has the value 1 if the argument is *true*, 0 otherwise. Prices and offers can assume only integer values in the range $\{1, \dots, P\}$.

The goal of the seller is to decide on a non-increasing sequence of prices that maximizes the profit.

1. Formulate the maximization problem as a recursive formula.
2. Write the pseudocode of a dynamic program that implements efficiently the recursive formula and the pseudocode of a program that finds an optimal sequence.
3. Discuss the time and the space complexity of the proposed solution.